

DOCUMENTO DE TECNOLOGIAS

RPA PARA CONSULTA PÚBLICA E ENVIO VIA WHATSAPP

Proposta tecnológica baseada no ecossistema Python

Documento	Tecnologias e justificativas técnicas
Versão	1.0
Data	22 de setembro de 2026

Objetivo

Definir uma stack enxuta, justificável e suficiente para implementar o fluxo completo do desafio: automatizar a consulta em um sistema público, extrair e estruturar dados, gerar uma mensagem personalizada, realizar o envio via WhatsApp e registrar o histórico da execução.

Stack principal

Python • FastAPI • Playwright • SQLAlchemy • SQLite • WhatsApp Cloud API

1. Stack tecnológica

A solução utiliza Python como linguagem central e separa automação, aplicação web, persistência, integração e testes em componentes independentes. A escolha prioriza simplicidade de desenvolvimento, rastreabilidade e facilidade de demonstração técnica.

Tecnologia	Papel	Uso no projeto
Python 3.12+	Linguagem principal	Centraliza o backend, automação, tratamento de dados e integrações em um único ecossistema.
FastAPI	Backend / API	Expõe os endpoints da aplicação, coordena o fluxo do RPA e fornece uma base simples para a interface web.
Uvicorn	Servidor ASGI	Executa a aplicação FastAPI durante o desenvolvimento e a demonstração.
Playwright	Automação web / RPA	Controla o navegador, preenche campos, aciona consultas, aguarda carregamentos e extrai os resultados.

Pydantic	Validação de dados	Valida entradas e estrutura os dados extraídos antes do processamento e da persistência.
SQLAlchemy	ORM / persistência	Organiza o acesso ao banco e evita espalhar consultas SQL pela aplicação.
SQLite	Banco de dados	Armazena histórico, status, mensagens e erros com configuração mínima para o MVP.
httpx	Cliente HTTP	Realiza chamadas assíncronas para serviços externos, especialmente a API do WhatsApp.
Meta WhatsApp Cloud API	Integração com WhatsApp	Realiza o envio oficial da mensagem para o número utilizado na demonstração.
Jinja2	Templates HTML	Renderiza a interface web diretamente pelo backend, evitando um frontend separado no MVP.
Bootstrap 5	Interface / responsividade	Fornece componentes visuais e responsividade suficientes para uma interface administrativa simples.
python-dotenv	Configuração	Carrega variáveis de ambiente sem manter tokens, credenciais e chaves no código-fonte.
logging	Logs	Registra etapas, erros e informações úteis para diagnóstico e rastreabilidade.
pytest	Testes	Valida regras de negócio, tratamento de dados e serviços sem depender exclusivamente de testes manuais.
Git + GitHub	Versionamento	Mantém histórico do código, facilita revisão e organiza a entrega do projeto.

2. Organização por responsabilidade

Aplicação web

FastAPI + Uvicorn + Jinja2 + Bootstrap 5 - Recebe a consulta do usuário, exibe os dados encontrados, permite gerar/enviar a mensagem e disponibiliza o histórico.

Automação e extração

Playwright - Executa a navegação no sistema público e coleta as informações diretamente da fonte consultada.

Validação e tratamento

Pydantic + Python - Normaliza os dados extraídos, valida campos obrigatórios e trata resultados ausentes ou incompletos.

Persistência

SQLAlchemy + SQLite - Registra a consulta, dados encontrados, data/hora, destinatário, mensagem, status e descrição de erros.

Comunicação externa

httpx + Meta WhatsApp Cloud API - Envia a mensagem gerada e captura o retorno da API para atualizar o status do envio.

Observabilidade e qualidade

logging + pytest - Mantém logs técnicos e permite testar regras críticas de forma repetível.

3. Fluxo técnico da solução

1. Usuário informa o dado de consulta na interface.
2. FastAPI inicia a execução e verifica se a operação já foi processada.
3. Playwright acessa o sistema público, realiza a consulta e extrai o resultado.
4. Pydantic valida e estrutura os dados obtidos.
5. SQLAlchemy registra a execução no SQLite.
6. O serviço de mensagens monta o texto personalizado.
7. httpx envia a mensagem pela Meta WhatsApp Cloud API.
8. A aplicação registra o status final do processamento e do envio.

4. Segurança e configuração

- **Variáveis de ambiente:** Tokens, chaves e identificadores da API do WhatsApp ficam em arquivo `.env`, que não deve ser versionado.
- **Arquivo `.env.example`:** Documenta apenas os nomes das variáveis necessárias, sem valores reais ou segredos.
- **Idempotência:** Um identificador único/hash da consulta e o status de envio impedem processamento ou disparo duplicado.
- **Tratamento de exceções:** Falhas de navegação, timeout, ausência de resultado, erro de extração e falha de envio são registradas e devolvidas de forma controlada.

5. Dependências principais

```
fastapi
uvicorn[standard]
playwright
pydantic
sqlalchemy
httpx
jinja2
python-dotenv
pytest
```

Observação: SQLite e o módulo logging fazem parte da solução, mas não precisam ser instalados como pacotes externos. Bootstrap pode ser carregado via CDN na interface.

Decisão arquitetural

A stack foi deliberadamente mantida enxuta para que o foco da entrega permaneça no processo automatizado, na qualidade da extração, na integração entre componentes, no tratamento de falhas e na rastreabilidade. Tecnologias adicionais de infraestrutura podem ser introduzidas posteriormente caso a solução evolua para produção.